CONTROL SYSTEMS Part-1
MATLAB
LABORATORY

Submitted By: Jit Debnath
Roll no: 24/249
ASTU Roll no: 2481101218

DEPARTMENT OF ELECTRICAL ENGINEERING
ASSAM ENGINEERING COLLEGE
JALUKBARI, GUWAHATI, ASSAM, INDIA
PIN:781013

LIST OF ASSIGNMENTS **PAGE**

***Copies of all the MATLAB Programs & Source code available on:**
https://github.com/ti8anACEx/matlab_control_systems_assignment

ASSIGNMENT 1

`1. Matrix and Array Operations

(a) For a square matrix A (say, a 3 x 3 matrix) test the following commands: A^2, A'(transpose A), A(2, 3), A(2, 2), A(1,:), A(:, 1), A(:, 1:2:3), A(2:1:3, 2:1:3), inv (A), det(A), diag(A), rank(A), expm(A), size(A), length (A), max(A), min(A)

MATLAB PROGRAM

```
A = [2 3 4;
 5 7 8;
 1 0 6]

A_squared = A^2
A_transpose = A'
A_2_3 = A(2,3)
A_2_2 = A(2,2)
A_row1 = A(1,:)
A_col1 = A(:,1)
A_col_1_3 = A(:,1:2:3)
A_submatrix = A(2:3,2:3)
A_inverse = inv(A)
A_determinant = det(A)
A_diagonal = diag(A)
A_rank = rank(A)
A_exponential = expm(A)
A_size = size(A)
A_length = length(A)
A_max = max(A)
A_min = min(A)
```

```
A =
     2     3     4
     5     7     8
     1     0     6
A_squared =
    23    27    56
    53    64   124
     8     3    40
A_transpose =
     2     5     1
     3     7     0
     4     8     6
A_2_3 =
     8
A_2_2 =
     7
A_row1 =
     2     3     4
A_col1 =
     2
     5
     1
A_col_1_3 =
     2     4
     5     8
     1     6
A_submatrix =
     7     8
     0     6
A_inverse =
   -4.2000    1.8000    0.4000
    2.2000   -0.8000   -0.4000
    0.7000   -0.3000    0.1000
A_determinant =
  -10.0000
```

```
A_diagonal =
     2
     7
     6
A_rank =
     3
A_exponential =
   1.0e+04 *
    0.5385    0.5411    1.6066
    1.2559    1.2632    3.7443
    0.1361    0.1328    0.4190
A_size =
     3     3
A_length =
     3
A_max =
     5     7     8
A_min =
     1     0     4
```

`1. Matrix and Array Operations

(b) Consider two square matrices A and B and test the following operations:
A + B, A – B, A* B, A .* B, A / B, A ./ B, [A B], [A' B']

MATLAB PROGRAM

```
B = [1 2 3;
 0 1 4;
 5 6 0]

A_plus_B = A + B
A_minus_B = A - B
A_multiply_B = A * B
A_elem_mult_B = A .* B
A_div_B = A / B
A_elem_div_B = A ./ B
A_concat = [A B]
A_trans_B_trans = [A' B']
```

```
B =
     1     2     3
     0     1     4
     5     6     0
A_plus_B =
     3     5     7
     5     8    12
     6     6     6
A_minus_B =
     1     1     1
     5     6     4
    -4    -6     6
A_multiply_B =
    22    31    18
    45    65    43
    31    38     3
A_elem_mult_B =
     2     6    12
     0     7    32
fx   5     0     0
```

```
A_div_B =
   -8.0000    7.0000    2.0000
  -20.0000   17.0000    5.0000
  -54.0000   42.0000   11.0000
A_elem_div_B =
    2.0000    1.5000    1.3333
       Inf    7.0000    2.0000
    0.2000         0       Inf
```

```
A_concat =
     2     3     4     1     2     3
     5     7     8     0     1     4
     1     0     6     5     6     0
A_trans_B_trans =
     2     5     1     1     0     5
     3     7     0     2     1     6
     4     8     6     3     4     0
```

2. Test the following commands:

Zeros(3), zeros(3, 4); ones(5), ones(2, 3), eye(4), eye(4, 6)

MATLAB PROGRAM

Z1 = zeros(3)
Z2 = zeros(3,4)

O1 = ones(5)
O2 = ones(2,3)

I1 = eye(4)
I2 = eye(4,6)

```
Z1 =
     0     0     0
     0     0     0
     0     0     0
Z2 =
     0     0     0     0
     0     0     0     0
     0     0     0     0
O1 =
     1     1     1     1     1
     1     1     1     1     1
     1     1     1     1     1
     1     1     1     1     1
     1     1     1     1     1
O2 =
     1     1     1
     1     1     1
I1 =
     1     0     0     0
     0     1     0     0
     0     0     1     0
     0     0     0     1
I2 =
     1     0     0     0     0     0
     0     1     0     0     0     0
     0     0     1     0     0     0
     0     0     0     1     0     0
```

3. **Solve for x in the equation Cx = D, where C=[1 2 1;2 3 2;-1 0 1] and D=[1; 1; 0]**

MATLAB PROGRAM

C = [1 2 1;
2 3 2;
-1 0 1]

D = [1; 1; 0]

*X_solution = C \ D % ie, C_inverse * D*

```
C =
     1     2     1
     2     3     2
    -1     0     1
D =
     1
     1
     0
X_solution =
   -0.5000
    1.0000
   -0.5000
```

4. **Common functions**
   a. **Consider, x = pi/3 and find sin(x), cos(x), tan(x), exp(x), log(x), sqrt(x), round(x)**
   b. **Consider, z = 3+4 * i (a complex number) and find abs(z), angle(z)**
   c. **Consider, a = 0.866 and find acos(a), asin(a), atan(a)**

MATLAB PROGRAM

```
% (a)
x = pi/3

sin_x = sin(x)
cos_x = cos(x)
tan_x = tan(x)
exp_x = exp(x)
log_x = log(x)
sqrt_x = sqrt(x)
round_x = round(x)

% (b)
z = 3 + 4i
abs_z = abs(z)
angle_z = angle(z) % = atan(4/3) in rad

% (c)
a = 0.866
acos_a = acos(a)
asin_a = asin(a)
atan_a = atan(a)
```

```
x =
    1.0472
sin_x =
    0.8660
cos_x =
    0.5000
tan_x =
    1.7321
exp_x =
    2.8497
log_x =
    0.0461
sqrt_x =
    1.0233
round_x =
     1
```

```
z =
   3.0000 + 4.0000i
abs_z =
     5
angle_z =
    0.9273
```

```
a =
    0.8660
acos_a =
    0.5236
asin_a =
    1.0471
atan_a =
    0.7137
```

5. **Consider two polynomials....**
   a. **Multiply three polynomials, x(s) = s4+3s3-2s2+5s+1, y(s) = 5s2+4s-2, z(s) = 5s3-6s+7**
   b. **Multiply polynomials: g(s) = (s+2)2, h(s) = s2-4s+3**

MATLAB PROGRAM

```
% a. Multiply three polynomials:
x = [1 3 -2 5 1]
y = [5 4 -2]
z = [5 0 -6 7]

xy_result = conv(x, y)
xyz_result = conv(xy_result, z)

% b. Multiply polynomials:
g_base = [1 2]
g_squared = conv(g_base, g_base)

h = [1 -4 3]
final_result = conv(g_squared, h)
```

```
n_result =
     3    14     9     4
x =
     1     3    -2     5     1
y =
     5     4    -2
z =
     5     0    -6     7
xy_result =
     5    19     0    11    29    -6    -2
xyz_result =
    25    95   -30   -24   278   -96  -107   239   -30   -14
g_base =
     1     2
g_squared =
     1     4     4
h =
     1    -4     3
final_result =
     1     0    -9    -4    12
```

6.Test the following:

- **x = [0:1:10]**
- **x'**
- **p = [1:0.001:1]**
- **p'**
- **If x = [0 pi/2 pi 3*pi/2 2*pi] find y = sin(x)**
- **If z = [0:pi/6:pi] find zz = cos (z) and zzz = cos(z/2)**

MATLAB PROGRAM

```
% x vector and its transpose
x1 = 0:1:10
x1_trans = x1'

% p vector and its transpose
p = 1:0.001:1
p_trans = p'

% Given x values and sin(x)
x2 = [0 pi/2 pi 3*pi/2 2*pi]
y_sin = sin(x2)

% z values and cosine operations
z = 0:pi/6:pi
zz_cos = cos(z)
zzz_cos = cos(z/2)
```

```
x1 =
     0     1     2     3     4     5     6     7     8     9    10
x1_trans =
     0
     1
     2
     3
     4
     5
     6
     7
     8
     9
    10
p =
     1
p_trans =
     1
x2 =
         0    1.5708    3.1416    4.7124    6.2832
y_sin =
         0    1.0000    0.0000   -1.0000   -0.0000
z =
         0    0.5236    1.0472    1.5708    2.0944    2.6180    3.1416
zz_cos =
    1.0000    0.8660    0.5000    0.0000   -0.5000   -0.8660   -1.0000
zzz_cos =
    1.0000    0.9659    0.8660    0.7071    0.5000    0.2588    0.0000
```

ASSIGNMENT 2

1.Partial Fraction Expansion

Test 1: Find the partial fraction expansions of the following using MATLAB:

$$F(s)=\frac{6}{s^3+6s^2+11s+6} \qquad F(s)=\frac{20s+20}{s^3+5s^2+7s+3} \qquad F(s)=\frac{s+2}{s(s+1)(s^2+9)}$$

$$F(s)=\frac{120(s+2)}{(s+1)^2(s+3)} \qquad F(s)=\frac{2}{(s+1)(s+3)^2}$$

MATLAB PROGRAM

```
num1 = [6]
den1 = [1 6 11 6]
[r1, p1, k1] = residue(num1, den1)

num2 = [120 240]
den2 = conv([1 1 1], [1 3])
[r2, p2, k2] = residue(num2, den2)

num3 = [20 20]
den3 = [1 5 7 3]
[r3, p3, k3] = residue(num3, den3)

num4 = [2]
den4 = conv([1 1], [1 3 3 9])
[r4, p4, k4] = residue(num4, den4)

num5 = [1 2]
den5_step1 = conv([1 0], [1 1])
den5 = conv(den5_step1, [1 0 9])
[r5, p5, k5] = residue(num5, den5)
```

```
num1 =
     6
den1 =
     1     6    11     6
r1 =
    3.0000
   -6.0000
    3.0000
p1 =
   -3.0000
   -2.0000
   -1.0000
k1 =
     []
```

```
num2 =
   120   240
den2 =
     1     4     4     3
r2 =
  -17.1429 + 0.0000i
    8.5714 -44.5384i
    8.5714 +44.5384i
p2 =
   -3.0000 + 0.0000i
   -0.5000 + 0.8660i
   -0.5000 - 0.8660i
k2 =
     []
```

```
num3 =
    20    20
den3 =
     1     5     7     3
r3 =
  -10.0000
   10.0000
    0.0000
p3 =
   -3.0000
   -1.0000
   -1.0000
k3 =
     []
```

```
num4 =
     2
den4 =
     1     4     6    12     9
r4 =
  -0.0833 + 0.0000i
  -0.0833 + 0.0000i
  -0.0833 - 0.0000i
   0.2500 + 0.0000i
p4 =
  -3.0000 + 0.0000i
  -0.0000 + 1.7321i
  -0.0000 - 1.7321i
  -1.0000 + 0.0000i
k4 =
     []
```

```
num5 =
     1     2
den5_step1 =
     1     1     0
den5 =
     1     1     9     9     0
r5 =
  -0.0611 + 0.0167i
  -0.0611 - 0.0167i
  -0.1000 + 0.0000i
   0.2222 + 0.0000i
p5 =
   0.0000 + 3.0000i
   0.0000 - 3.0000i
  -1.0000 + 0.0000i
   0.0000 + 0.0000i
k5 =
     []
```

2.Determination of zeros & poles of transfer function

Command to determine zeros (z), poles (p) and gain (k) is

[z, p, k] = tf2zp(num, den)

Where num and den are the matrices, elements of which are the coefficients of numerator and denominator polynomial respectively.

For the transfer function, , type the following in command window

>>num=[0 020 20]; den=[1 5 7 3]; [z,p,k]=tf2zp(num,den) <press enter>

This will produce output as z = -1, p = -3, -1, -1, k = 20

MATLAB PROGRAM

```
num6 = [20 20]
den6 = [1 5 7 3]

[z6, p6, k6] = tf2zp(num6, den6)
```

```
num6 =
    20    20
den6 =
     1     5     7     3
z6 =
    -1
p6 =
  -3.0000 + 0.0000i
  -1.0000 + 0.0000i
  -1.0000 - 0.0000i
k6 =
    20
```

3.Determination of transfer function from poles, zeros and gain

Type the following set of commands and see the results:

>> z = [-1]; p = [-3; -1; -1]; k = 20; [num, den] = zp2tf(z, p, k); printsys(num, den, ‘s’) <press enter>

MATLAB PROGRAM

```
z7 = [-1]
p7 = [-3 -1 -1]
k7 = 20

[num7, den7] = zp2tf(z7, p7, k7)
printsys(num7, den7, 's')
```

```
z7 =
    -1
p7 =
    -3    -1    -1
k7 =
    20
num7 =
     0     0    20    20
den7 =
     1     5     7     3

num/den =

          20 s + 20
   ---------------------
   s^3 + 5 s^2 + 7 s + 3
```

Test 2

(a) Find poles, zeros and gain of all the problems given in Test 1, ie:

$$F(s)=\frac{6}{s^3+6s^2+11s+6}$$

$$F(s)=\frac{120(s+2)}{(s+1)^2(s+3)}$$

$$F(s)=\frac{20s+20}{s^3+5s^2+7s+3}$$

$$F(s)=\frac{2}{(s+1)(s+3)^2}$$

$$F(s)=\frac{s+2}{s(s+1)(s^2+9)}$$

MATLAB PROGRAM

```
num1 = [6]
den1 = [1 6 11 6]
[z1, p1, k1] = tf2zp(num1, den1)

num2 = [120 240]
den2 = conv([1 1 1], [1 3])
[z2, p2, k2] = tf2zp(num2, den2)

num3 = [20 20]
den3 = [1 5 7 3]
[z3, p3, k3] = tf2zp(num3, den3)

num4 = [2]
den4 = conv([1 1], [1 3 3 9])
[z4, p4, k4] = tf2zp(num4, den4)

num5 = [1 2]
den5_step1 = conv([1 0], [1 1])
den5 = conv(den5_step1, [1 0 9])
[z5, p5, k5] = tf2zp(num5, den5)
```

```
num1 =
     6
den1 =
     1     6    11     6
z1 =
  0×1 empty double column vector
p1 =
   -3.0000
   -2.0000
   -1.0000
k1 =
     6
num2 =
   120   240
den2 =
     1     4     4     3
z2 =
    -2
p2 =
  -3.0000 + 0.0000i
  -0.5000 + 0.8660i
  -0.5000 - 0.8660i
k2 =
   120
num3 =
    20    20
den3 =
     1     5     7     3
z3 =
    -1
p3 =
  -3.0000 + 0.0000i
  -1.0000 + 0.0000i
  -1.0000 - 0.0000i
k3 =
    20
num4 =
     2
den4 =
     1     4     6    12     9
z4 =
  0×1 empty double column vector
p4 =
  -3.0000 + 0.0000i
  -0.0000 + 1.7321i
  -0.0000 - 1.7321i
  -1.0000 + 0.0000i
k4 =
     2
num5 =
     1     2
den5_step1 =
     1     1     0
den5 =
     1     1     9     9     0
z5 =
    -2
p5 =
   0.0000 + 0.0000i
   0.0000 + 3.0000i
   0.0000 - 3.0000i
  -1.0000 + 0.0000i
k5 =
     1
```

Test 2

(b) For the following poles, zeros and gain, determine transfer functions:

(i) zeros at s = -1, -2; poles at s = 0, -4, -6 and gain k = 5

(ii) zeros at s = 1; poles at s = 0, -2, -1+j0.5, -1-j0.5 and gain k = 10

MATLAB PROGRAM

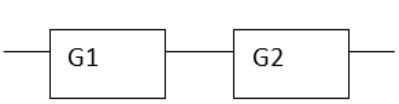
```
% (i)
z6 = [-1; -2]
p6 = [0; -4; -6]
k6 = 5

[num6, den6] = zp2tf(z6, p6, k6)

% (ii)
z7 = [-1]
p7 = [0; -2; -1+0.5i; -1-0.5i]
k7 = 10

[num7, den7] = zp2tf(z7, p7, k7)
```

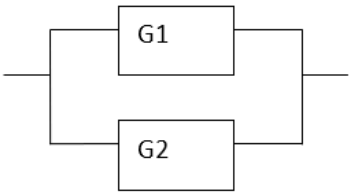
```
z6 =
    -1
    -2
p6 =
     0
    -4
    -6
k6 =
     5
num6 =
     0     5    15    10
den6 =
     1    10    24     0
```

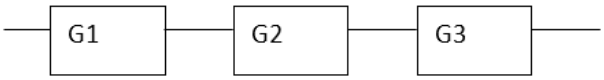
```
z7 =
    -1
p7 =
   0.0000 + 0.0000i
  -2.0000 + 0.0000i
  -1.0000 + 0.5000i
  -1.0000 - 0.5000i
k7 =
    10
num7 =
     0     0     0    10    10
den7 =
    1.0000    4.0000    5.2500    2.5000         0
>>
```

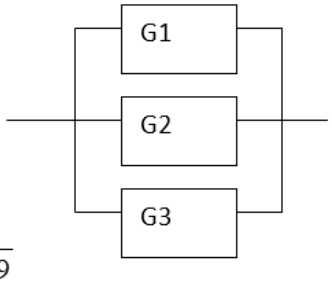
Test 3

1. Find the transfer functions for the following block diagrams:









where $G1 = \frac{10}{s}$ $G2 = \frac{s+1}{s(s+5)}$ $G3 = \frac{10}{s^2 + 3s + 9}$

MATLAB PROGRAM

G1 = tf([10], [1 0])
G2 = tf([1 1], [1 5 0])
G3 = tf([10], [1 3 9])

series_G1_G2 = series(G1, G2)
parallel_G1_G2 = parallel(G1, G2)
series_all = series(series_G1_G2, G3)
parallel_all = parallel(parallel_G1_G2, G3)

```
Continuous-time transfer function.
Model Properties
```

```
G1 =

  10
  --
  s
```

```
G2 =

    s + 1
  ---------
  s^2 + 5 s
```

```
G3 =

        10
  -------------
  s^2 + 3 s + 9
```

```
series_G1_G2 =

   10 s + 10
  -----------
  s^3 + 5 s^2
```

```
parallel_G1_G2 =

   11 s^2 + 51 s
  -------------
   s^3 + 5 s^2
```

```
series_all =

             100 s + 100
  ----------------------------
  s^5 + 8 s^4 + 24 s^3 + 45 s^2
```

```
parallel_all =

  11 s^4 + 94 s^3 + 302 s^2 + 459 s
  ---------------------------------
    s^5 + 8 s^4 + 24 s^3 + 45 s^2
```

Test 3

2. Find the closed loop transfer functions of negative feedback systems:

(a) $G1(s)=\frac{100}{s(s+4)}$ and $H1(s)=\frac{1}{s+1}$ (b) $G2(s)=\frac{5(s+3)}{s^2+3s+9}$ and $H2(s)=\frac{2}{s}$

MATLAB PROGRAM

```
G4 = tf([100], [1 4 0])
H4 = tf([1], [1 1])

CLTF1 = feedback(G4, H4)

G5 = tf([5 15], [1 3 9])
H5 = tf([2], [1 0])

CLTF2 = feedback(G5, H5)
```

```
G4 =

      100
  ---------
  s^2 + 4 s

H4 =

    1
  -----
  s + 1

Continuous-time transfer function.
Model Properties

CLTF1 =

        100 s + 100
  ----------------------
  s^3 + 5 s^2 + 4 s + 100

Continuous-time transfer function.
Model Properties

G5 =

    5 s + 15
  -------------
  s^2 + 3 s + 9

Continuous-time transfer function.
Model Properties

H5 =

  2
  -
  s

Continuous-time transfer function.
Model Properties


CLTF2 =

       5 s^2 + 15 s
  ----------------------
  s^3 + 3 s^2 + 19 s + 30

Continuous-time transfer function.
Model Properties
```

ASSIGNMENT 3

Q.1. Write a MATLAB program to study the step response of a first order system and second order system.

Time Domain analysis of a standard 2nd order LTI system

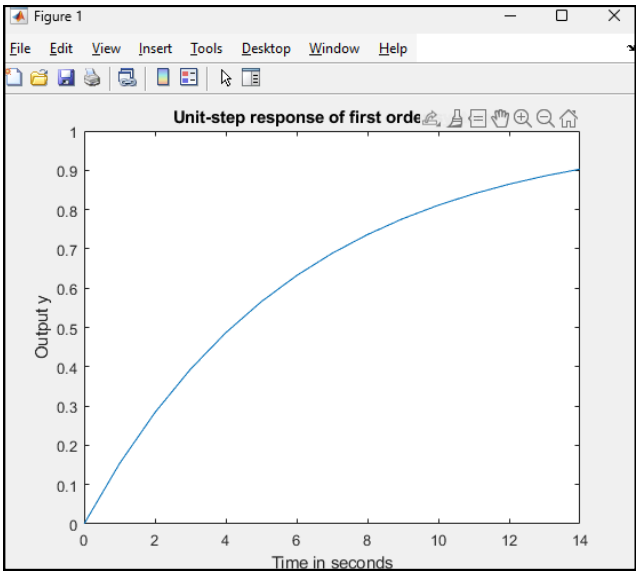
Simulate the following system using MATLAB.

$$R(s) \rightarrow \boxed{\frac{K\omega_n^2}{s^2+2\zeta\omega_n s+\omega_n^2}} \rightarrow Y(s)$$

MATLAB PROGRAM

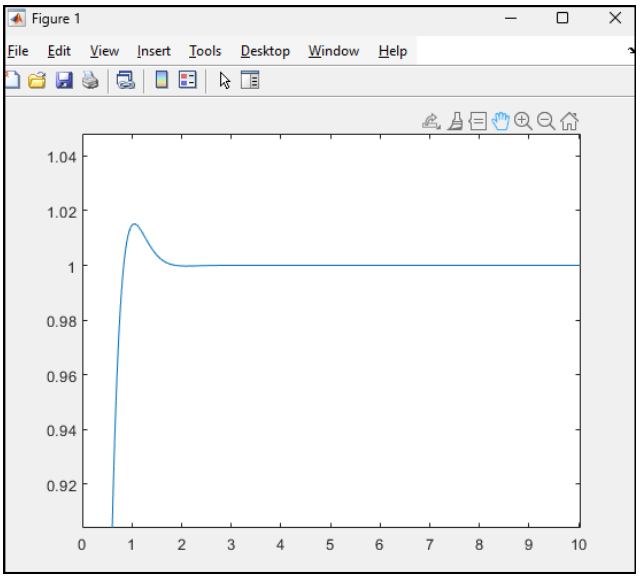
```
% Part A: Step Response of a First-Order System
clear;
clc;
T=input('Enter value of time constant T:');
% Define the Transfer Function: G(s) = 1 / (Ts + 1)
num=[1];
den=[T 1];
t=0:1:14;
y=step(num,den,t);
plot(t,y);
xlabel('Time in seconds');
ylabel('Output y');
title('Unit-step response of first order system');
```

```
Enter value of time constant T:6
```




```
% Part B: Step Response of a Second-Order System
clear; clc;
wn = input('Enter natural frequency (wn): ');
zeta = input('Enter damping ratio (zeta): ');
K = 1; % Assuming Gain K = 1
num = [K * wn^2];
den = [1, 2*zeta*wn, wn^2];
sys = tf(num, den);

t = 0:0.01:100;
step(sys, t);
y=step(sys, t);
plot(t,y);
```

```
Enter natural frequency (wn): 5
Enter damping ratio (zeta): 0.8
```



Q.2. Write a MATLAB program to plot the following: ***15***

a) Rise time (t_r) vs ζ

b) Peak time (t_p) vs ζ

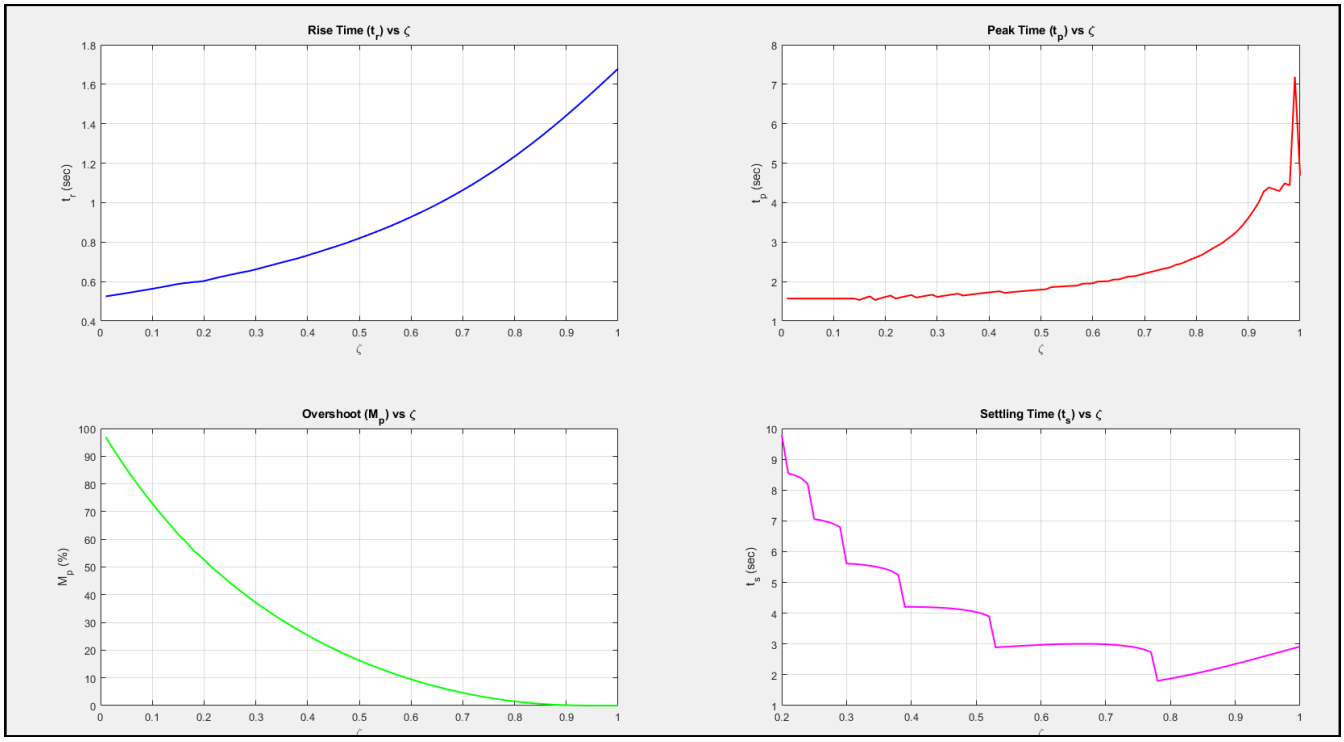
c) Overshoot (M_p) vs ζ

d) Settling time (t_s) vs ζ

by varying the damping ratio ζ between 0 to 1 (in case of a), b), and c)) and between 0.2 to 1 (in case of d)), keeping $\omega_n = 2$ rad/s, $K = 1$ for a unit step input.

MATLAB PROGRAM

```
% Constants given
wn = 2;
K = 1;

%% Part a, b, c: Zeta from 0 to 1
zeta_range1 = 0:0.01:1;
tr = []; tp = []; Mp = [];

for z = zeta_range1
 num = [K * wn^2];
 den = [1, 2*z*wn, wn^2];
 sys = tf(num, den);

 info = stepinfo(sys);
 tr = [tr, info.RiseTime];
 tp = [tp, info.PeakTime];
 Mp = [Mp, info.Overshoot];
end

%% Part d: Settling Time (Zeta from 0.2 to 1)
zeta_range2 = 0.2:0.01:1;
ts = [];

for z = zeta_range2
 num = [K * wn^2];
 den = [1, 2*z*wn, wn^2];
 sys = tf(num, den);

 info = stepinfo(sys);
 ts = [ts, info.SettlingTime];
end

%% Plotting the results
figure;

% a) Rise Time vs Zeta
subplot(2,2,1);
plot(zeta_range1, tr, 'b', 'LineWidth', 1.5);
title('Rise Time (t_r) vs \zeta');
xlabel('\zeta'); ylabel('t_r (sec)'); grid on;

% b) Peak Time vs Zeta
subplot(2,2,2);
plot(zeta_range1, tp, 'r', 'LineWidth', 1.5);
title('Peak Time (t_p) vs \zeta');
xlabel('\zeta'); ylabel('t_p (sec)'); grid on;

% c) Overshoot vs Zeta
subplot(2,2,3);
plot(zeta_range1, Mp, 'g', 'LineWidth', 1.5);
title('Overshoot (M_p) vs \zeta');
xlabel('\zeta'); ylabel('M_p (%)'); grid on;

% d) Settling Time vs Zeta
subplot(2,2,4);
plot(zeta_range2, ts, 'm', 'LineWidth', 1.5);
title('Settling Time (t_s) vs \zeta');
xlabel('\zeta'); ylabel('t_s (sec)'); grid on;
```



Q.3. Write a MATLAB program to plot the following:

a) Rise time vs

b) Peak time (t_p) vs

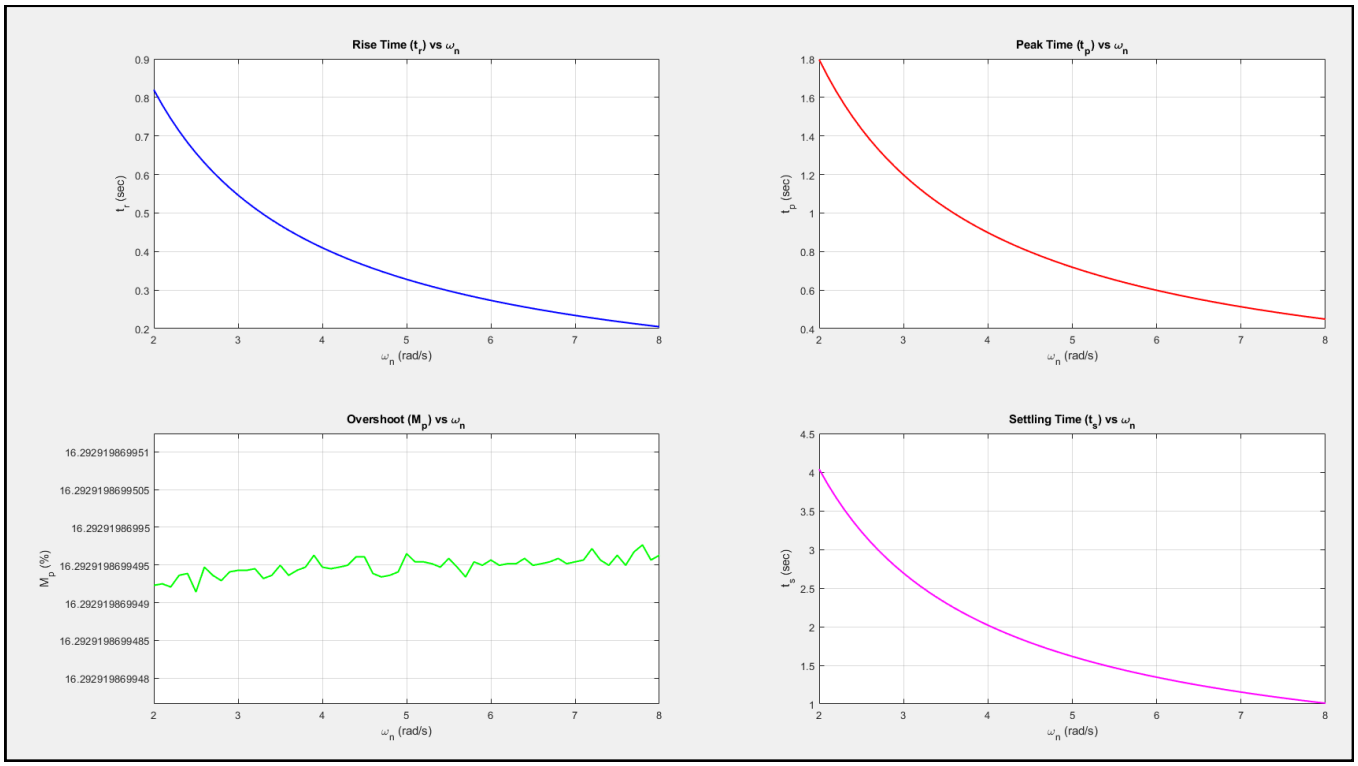
c) Overshoot (M_p) vs

d) Settling time (t_s) vs

by varying the natural freq. between 2 to 8rad/s, keeping zeta = 0.5 and K=1 for a unit step input.

MATLAB PROGRAM

```
% Constants given
zeta = 0.5;
K = 1;

% Range for wn: 2 to 8 rad/s
wn_range = 2:0.1:8;

% Initialize empty vectors
tr = []; tp = []; Mp = []; ts = [];

for wn = wn_range
 num = [K * wn^2];
 den = [1, 2*zeta*wn, wn^2];
 sys = tf(num, den);
 info = stepinfo(sys);

 tr = [tr, info.RiseTime];
 tp = [tp, info.PeakTime];
 Mp = [Mp, info.Overshoot];
 ts = [ts, info.SettlingTime];
end

%% Plotting the results
figure;

subplot(2,2,1);
plot(wn_range, tr, 'b', 'LineWidth', 1.5);
title('Rise Time (t_r) vs \omega_n');
xlabel('\omega_n (rad/s)'); ylabel('t_r (sec)'); grid on;

subplot(2,2,2);
plot(wn_range, tp, 'r', 'LineWidth', 1.5);
title('Peak Time (t_p) vs \omega_n');
xlabel('\omega_n (rad/s)'); ylabel('t_p (sec)'); grid on;

subplot(2,2,3);
plot(wn_range, Mp, 'g', 'LineWidth', 1.5);
title('Overshoot (M_p) vs \omega_n');
xlabel('\omega_n (rad/s)'); ylabel('M_p (%)'); grid on;

subplot(2,2,4);
plot(wn_range, ts, 'm', 'LineWidth', 1.5);
title('Settling Time (t_s) vs \omega_n');
xlabel('\omega_n (rad/s)'); ylabel('t_s (sec)'); grid on;
```



3. Justify each of the plots of the simulated results from the theoretical point of view.

ANSWER

Justification of the Plots:

- **Rise Time (tr) and Peak Time (tp):** In the formula, ωn is in the denominator:

$$t_p = \frac{\pi}{\omega_n \sqrt{1-\zeta^2}}$$

As ωn increases, the denominator gets larger, making the time values smaller. This means the system becomes faster. Plots should show a downward curve (decay).

- **Overshoot (Mp):**

$$M_p = e^{-\left(\frac{\zeta\pi}{\sqrt{1-\zeta^2}}\right)} \times 100\%$$

There is no ωn in this formula. Since ζ is constant (0.5), the overshoot stays exactly the same regardless of the frequency. Plot should be a perfectly horizontal line.

- **Settling Time (ts):** For a 2% tolerance, the formula is:

$$t_s \approx \frac{4}{\zeta\omega_n}$$

Again, ωn is in the denominator. A higher natural frequency means the oscillations die out much quicker. Plot should show a downward curve.

Conclusion:

1. **Speed**: tr,tp, and ts all decrease (the system gets faster).
2. **Height**: Mp stays constant (the "relative bounce" doesn't change).

Test 1: Plot the root locus of the open-loop transfer function

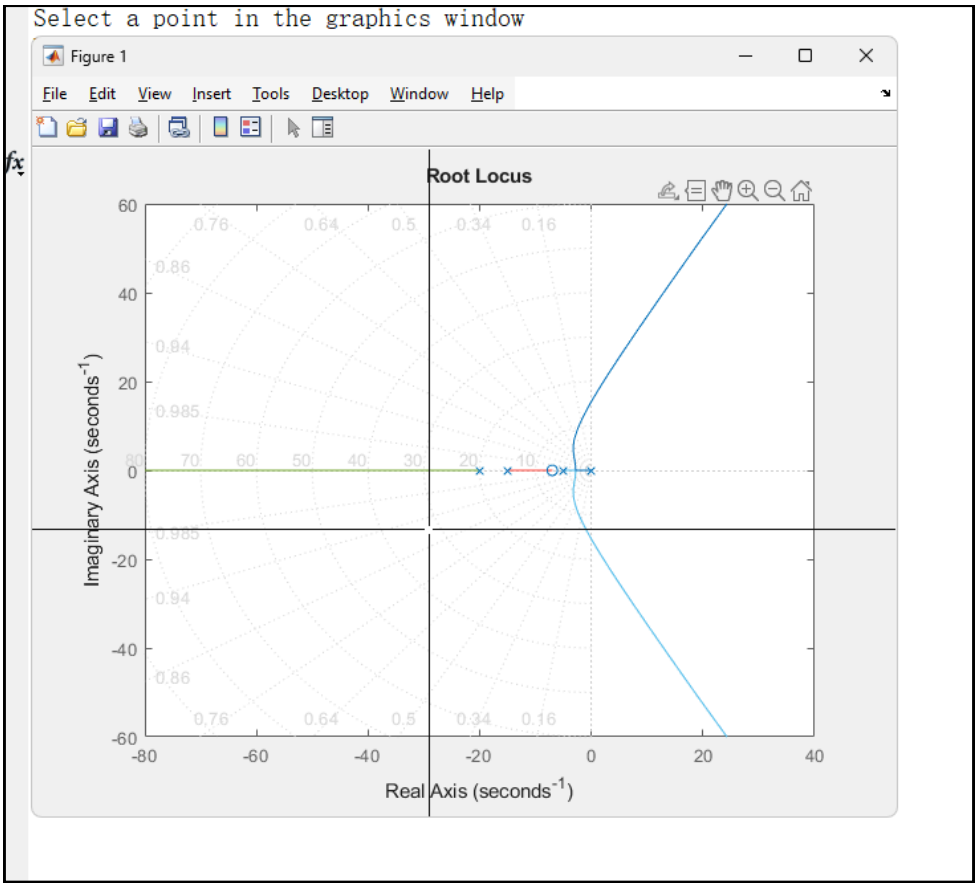
$$G(s) = \frac{(s+7)}{s(s+5)(s+15)(s+20)}$$

Obtain the value of gain, pole location and %overshoot for a damping ratio 0.6 and natural frequency 6 rad/sec.

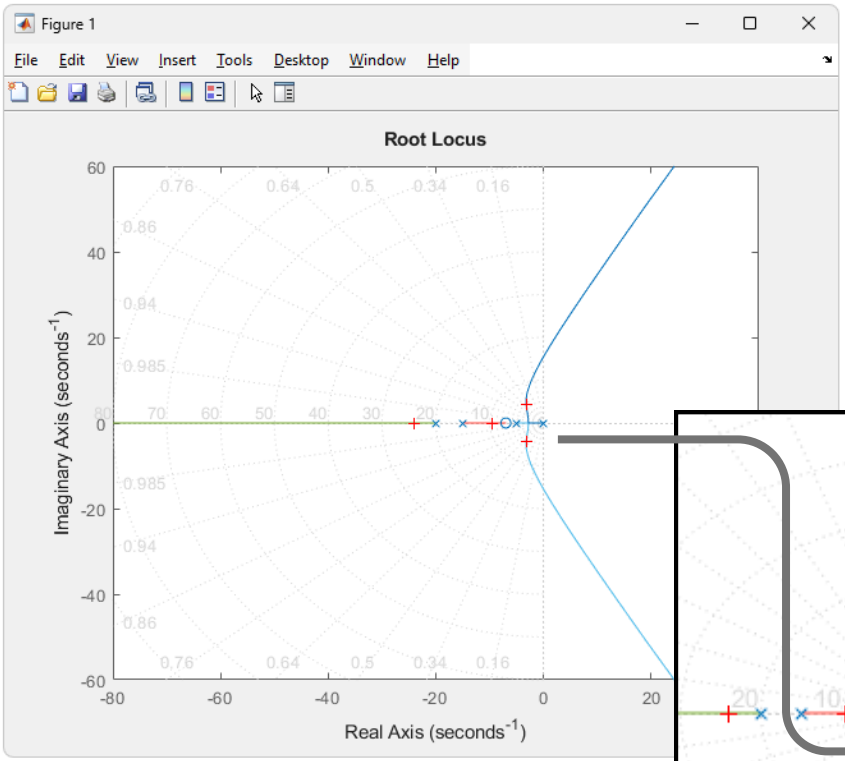
MATLAB PROGRAM

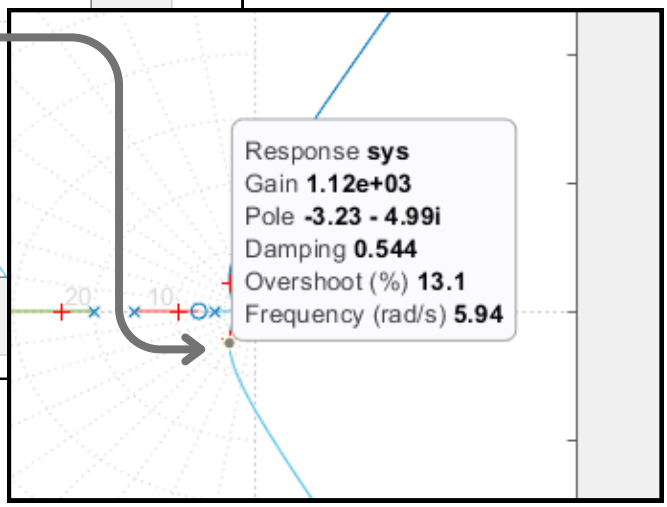
```
% Define the numerator (s + 7)
num = [1 7];

% Define the denominator s(s+5)(s+15)(s+20) using nested conv
den = conv([1 0], conv([1 5], conv([1 15], [1 20])));

% Create the Transfer Function
sys = tf(num, den);

% Plotting the Root Locus
figure;
rlocus(sys);
title('Root Locus of the Open-Loop Transfer Function');
grid on;

% Requirements
zeta = 0.6;
wn = 6;

% Calculating the theoretical coordinates (Target Bullseye)
sigma = -zeta * wn;
wd = wn * sqrt(1 - zeta^2);
desired_pole = sigma + 1j * wd;

% Interactive Gain Selection
% INSTRUCTIONS: When the plot opens, click where the locus intersects the zeta=0.6 line
[k, poles] = rlocfind(sys);

% Calculate theoretical overshoot based on zeta
overshoot = exp(-pi * zeta / sqrt(1 - zeta^2)) * 100;

% Print Results to Command Window
fprintf('Gain (K): %.4f\n', k);
fprintf('Theoretical Pole Location: %.4f + %.4fj\n', real(desired_pole), imag(desired_pole));
fprintf('Percentage Overshoot: %.2f%%\n', overshoot);
```

```
Select a point in the graphics window
```




```
selected_point =
  -3.1318 - 4.4743i
Gain (K): 991.0256
Theoretical Pole Location: -3.6000 + 4.8000j
Percentage Overshoot: 9.48%
>>
```



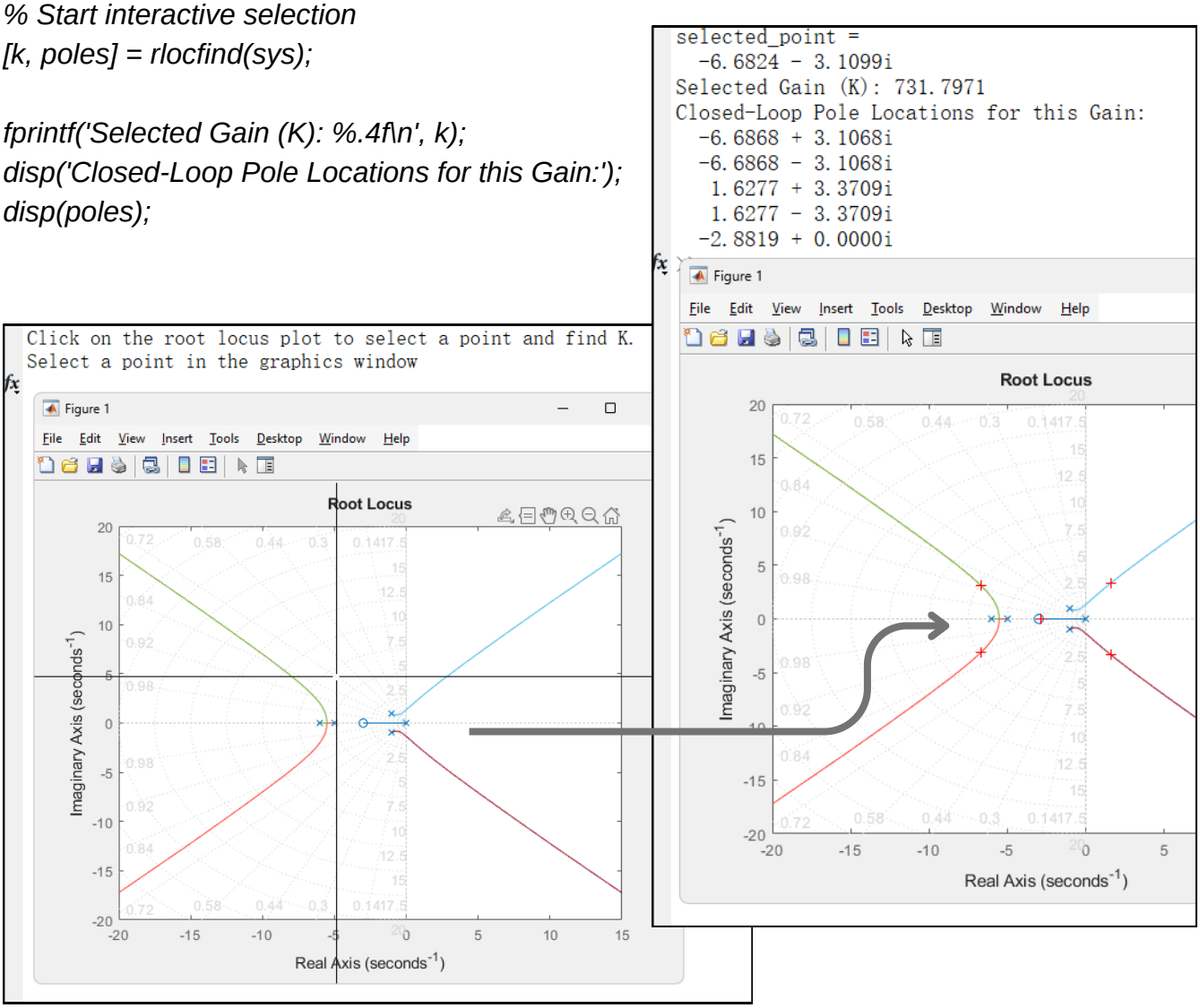


Test 2: Create an M-file to plot the root locus of the open loop transfer function

Test rlocfind command in this locus. $$G(s) = \frac{K(s+3)}{s(s+5)(s+6)(s^2+2s+2)}$$

MATLAB PROGRAM

```
% Define the numerator and Denominator
num = [1 3];
den = conv([1 0], conv([1 5], conv([1 6], [1 2 2])));
sys = tf(num, den);

% Plotting the Root Locus
figure;
rlocus(sys);
title('Root Locus of the Open-Loop Transfer Function - Test 2');
grid on;

disp('Click on the root locus plot to select a point and find K.');

% Start interactive selection
[k, poles] = rlocfind(sys);

fprintf('Selected Gain (K): %.4f\n', k);
disp('Closed-Loop Pole Locations for this Gain:');
disp(poles);
```

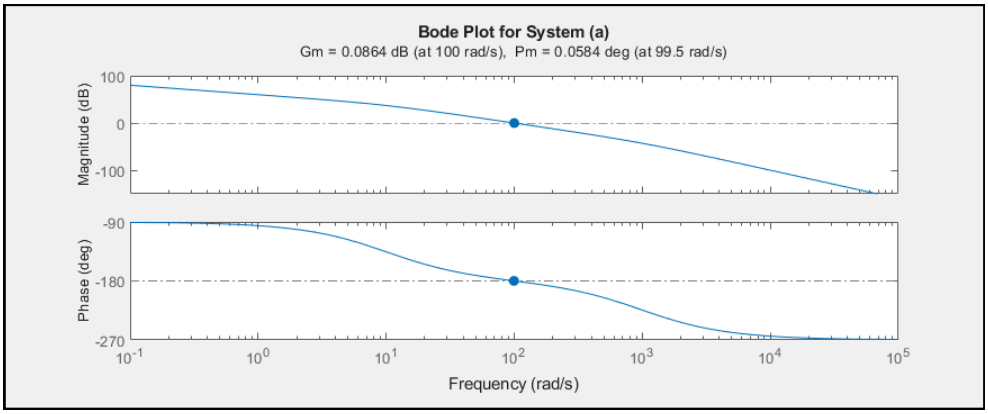


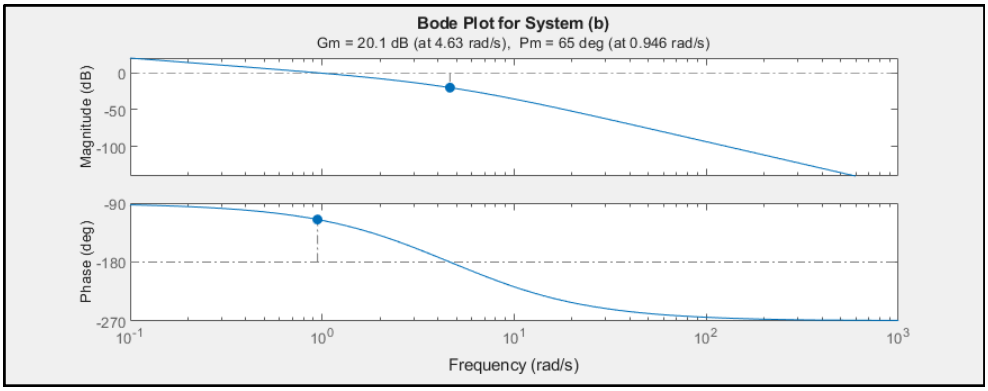
Test 1: Sketch the Bode plot for the transfer functions

$$G(s) = \frac{1000}{s(1+0.1s)(1+0.001s)} \qquad G(s) = \frac{15}{s(s+3)(0.7s+5)}$$

MATLAB PROGRAM

```
% System (a)
num_a = [1000];
den_a = [0.0001 0.101 1 0]; % Coefficients of 0.0001s^3 + 0.101s^2 + s
sys_a = tf(num_a, den_a);

% System (b)
num_b = [15];
den_b = [0.7 7.1 15 0]; % Coefficients of 0.7s^3 + 7.1s^2 + 15s
sys_b = tf(num_b, den_b);

% Plotting with Margins (Stability Info)
figure;
subplot(2,1,1);
margin(sys_a); % 'margin' is better than 'bode' because it shows Gain/Phase margins
title('Bode Plot for System (a)');

subplot(2,1,2);
margin(sys_b);
title('Bode Plot for System (b)');
```





Test 2: Sketch the Nyquist plot for the transfer functions

$$G(s) = \frac{1}{s^3 + 0.3s^2 + 5s + 1} \qquad G(s) = \frac{K(s+1)(s+3+7i)(s+3-7i)}{(s+1)(s+3)(s+5)(s+3+7i)(s+3-7i)}$$

MATLAB PROGRAM

```
% System (a)
num_2a = [1];
den_2a = [1 0.3 5 1];
sys_2a = tf(num_2a, den_2a);

% System (b) - Simplified
K = 1;
num_2b = [K];
den_2b = [1 8 15]; % (s+3)(s+5)=s^2 + 8s + 15
sys_2b = tf(num_2b, den_2b);

% Plotting Nyquist
figure;
subplot(1,2,1);
nyquist(sys_2a);
title('Nyquist Plot (a)');
grid on;

subplot(1,2,2);
nyquist(sys_2b);
title('Nyquist Plot (b)');
grid on;
```